Récursivité

Si on regarde la définition de la récursivité dans un dictionnaire on trouve :
Larousse : "Propriété que possède une règle ou un élément constituant de pouvoir **se répéter** de manière théoriquement indéfinie"

On peut retenir de cette définition l'idée de répétition qui est une première caractéristique de la récursivité, mais le point le plus important tient dans le "**se**" (de "**se répéter**"), qui est ici un pronom réfléchi, ce qui signifie que c'est la règle (dont parle la définition) qui possède le pouvoir d'entretenir la répétition.

Ce que cela signifie, c'est qu'une règle est récursive si elle fait appel à elle même pour se décrire. Du point de vue programmation, un algorithme est récursif s'il se contient lui-même dans sa définition, s'il se définit par lui même.

Si on regarde le point de vue d'un programme, en C++ par exemple on peut dire que pour pouvoir écrire l'appel d'une fonction, le compilateur a besoin de connaître son en-tête (nom de la fonction, type de retour, nombre d'argument et leurs types), ce qui est respecté à l'intérieur d'une fonction puisque toutes ces définitions ont été faites au début de la fonction, et donc il est tout à fait envisageable d'appeler une fonction depuis elle-même. Cette façon de faire s'appelle donc la **récursivité**.

Exemple 1 : la fonction factorielle

La factorielle d'un entier naturel N, notée $N!$, est définie comme étant le produit de tous les nombres entiers positifs qui lui sont inférieurs.

On la définit donc par la formule :

(1) $$N! = \prod_{i=1}^{N} i = 1\times 2\times 3\times ...\times (N-1)\times N$$

et en constatant que :

$$(N-1)! = \prod_{i=1}^{N-1} i = 1\times 2\times 3\times ...\times (N-1)$$

il vient tout naturellement la formule :

(2) $$N! = N\times (N-1)!$$

avec le cas particulier :

(2bis) $$1! = 1$$

et on ajoute (par convention)

(2ter) $$0! = 1$$

La programmation de cette fonction suivant la formule (1) se fait avec une boucle :

```
int Fact(int N)
{
  int i, F;

  for (i = 1, F = 1 ;i < N;i++)   // Oui on peut commencer à i = 2 !
    F = F * i;
  return F ;
```

```
}
```

En utilisant les formules (2) et (2ter) il vient naturellement une version récursive :

```
int Fact_Recursive(int N)
{
  if (N == 0)
    return 1;
  else            // Facultatif
    return N * Fact_Recursive(N - 1);
}
```

Le calcul de *N!* fait dans ce cas appel au calcul de *(N – 1) !*

On peut constater que l'appel récursif de la fonction joue ici le même rôle que la boucle dans la version itérative.

Exemple de fonctionnement : exécution de `Fact_Recursive(4)`

L'exécution de la fonction se fera par son appel par exemple ainsi :

```
…
int n ;
…
n = 4 ;
…
cout << n << "! = " << Fact-Recursive(n) << endl;
```

La fonction est tout d'abord exécutée avec pour argument la valeur **4**, qui est différente de **0**,
c'est donc la valeur de l'expression : `4 * Fact_Recursive(4 - 1)` qui doit être retournée,
ce qui provoque l'appel de : `Fact_Recursive(3)` (a)
ce qui provoque l'appel de : `Fact_Recursive(2)` (b)
ce qui provoque l'appel de : `Fact_Recursive(1)` (c)
ce qui provoque l'appel de : `Fact_Recursive(0)`
qui retourne **1**
la valeur de (c) est déterminée : `1 * 1 = 1`
la valeur de (b) est déterminée : `2 * 1 = 2`
la valeur de (c) est déterminée : `3 * 2 = 6`
la valeur du premier appel est enfin déterminée : `4 * 6 = 24`
et c'est cette valeur qui sera finalement affichée.

Contraintes de fonctionnement – Comment ça marche

Pour que l'exécution se déroule conformément à nos attentes, certaines conditions doivent être réunies :

- à la fin d'un appel de la fonction, le programme doit se retrouver et se continuer juste après cet appel,
- l'argument formel **N** d'un appel de la fonction doit conserver sa valeur jusqu'à la fin de l'appel, ce qui implique que chaque instance (chaque exemplaire de l'appel) de la fonction possède son propre exemplaire de N. Ce qui exactement la définition d'une variable locale.

Ces contraintes sont respectées par le C/C++. On peut dire que ces langages sont récursifs ou que ces langages supportent la récursivité.

Point technique

Les compilateurs C/C++ utilisent, pour ranger les adresses de retour et les variables locales (y compris les arguments passés par valeur) des fonctions, un espace mémoire avec une organisation « en pile », c'est à dire que :

- à chaque appel l'adresse de retour et les variables locales sont rangées à une extrémité de cet espace,

– à chaque fin de fonction, l'adresse de retour et les variables locales sont sorties de la même extrémité, dans l'ordre inverse du rangement.

Ainsi l'ordre de retour est bien conforme à l'inverse de l'ordre des appels.

Récursivité mutuelle ou récursivité croisée

En pratique, la récursivité est souvent supportée par une fonction qui s'appelle elle même, mais ça n'est pas une obligation. On peut rencontrer 2 fonctions qui s'appellent mutuellement, ou bien sûr plus de 2.

Exemple 2 : Pair/Impair

Exemple élémentaire de récursivité croisée :

Soit N un entier positif ou nul, on a les 2 définitions :

N est pair est vrai si N = 0
ou est équivalent à la proposition N – 1 est impair

et

N est impair est faux si N = 0
ou est équivalent à la proposition N – 1 est pair

De manière plus fonctionnelle :

Pour N >= 0

Pair (N)	= vrai	si N = 0
	= Impair(N – 1)	si N > 0

et

Impair (N)	= faux	si N = 0
	= Pair(N – 1)	si N > 0

Programmation :

```
#include <iostream>

using namespace std;

bool Impair(int N);

bool Pair(int N)
{
  if (0 == N)
    return true;
  else
    return Impair(N - 1);
}

bool Impair(int N)
{
  if (0 == N)
    return false;
  else
    return Pair(N - 1);
}

int main()
{
  int N;

  while (true)
  {
```

```
    cin >> N;
    if (Pair(N))
      cout << "Pair" << endl;
    else
      cout << "ImPair" << endl;
  }
}
```

Notez la déclaration de l'en-tête de la fonction Impair avant la déclaration de Pair.
Cela est nécessaire pour que le compilateur connaisse la fonction et puisse vérifier que son appel est correct.

Exemple 3 : la fonction puissance

L'élévation à une puissance entière positive d'un nombre réel est défini par la formule :

(3) $$x^N = \underbrace{x*x*x*...*x}_{N\,exemplaires\,de\,x}$$

En faisant les mêmes constatations que précédemment on peut écrire :

(4) $$x^N = x*x^{N-1}$$

et en ajoutant la convention usuelle :

(5) $$x^0 = 1$$

on obtient la fonction :

```
double Puissance-Recursive(double X, int N)
{
  if (N == 0)
    return 1 ;
  else
    return X * Puissance-Recursive(x, N - 1);
}
```

Intérêt de la récursivité

La question peut se poser de l'intérêt de la récursivité dans un cadre de programmation, on voit bien avec le premier exemple que les versions récursives et non récursives se valent en termes de longueur d'écriture, la version récursive étant peut-être un peu plus claire ou lisible.

En termes d'efficacité, la récursivité a un coût. En temps car l'appel d'une fonction n'est pas une opération instantanée, et le retour non plus. En occupation mémoire du fait du stockage de l'adresse de retour de la fonction et de la multiplication des variables locales. Enfin la mémoire disponible pour le stockage des adresses de retour et des variables locales (la pile) n'est pas extensible à l'infini et donc le nombre d'appel récursif est limité.

Il existe toutefois des situations où la programmation récursive est plus simple, notamment parce qu'elle correspond mieux à la définition que l'on a du problème et est dons plus rapide à programmer, plus lisible et plus simple à maintenir.

Récursivité terminale ou non terminale

Qu'est ce donc ?

On parle de récursivité terminale si l'appel récursif constitue la dernière opération de l'algorithme (de la fonction).

Exemple simple (élémentaire) :

Addition récursive (au sens de l'arithmétique de Péano)

IIIII + III = IIIII III
plus précisément
IIIII + III = IIIII I + II = IIIII I I + I = IIIII I I I + 0 = IIIIIIII

A + B = (A + 1) + (B - 1) jusqu'à ce qu'il n'y ait plus de B

ou avec une belle formule récursive :

$$Addition(A,B)=\left\{\begin{matrix} A \quad si \quad B=0 \\ Addition((A+1),(B-1)) \quad sinon \end{matrix}\right\}$$

Programmation :

```
int AddR1(int A, int B)
{
  if (B == 0)
    return A;
  else
  {
    return AddR1(A + 1, B - 1);
  }
}
```

qui correspond bien à la définition donnée. Si on la décompose on a bien :

```
int AddR1bis(int A, int B)
{
  int R;
  int Rm1;

  if (B == 0)
    R = A;
  else
  {
    A = A + 1;
    B = B - 1;
    R = AddR1bis(A, B);
  }
  return R;
}
```

Attention on peut être tenté d'écrire :

```
int AddR2(int A, int B)
{
  if (B == 0)
    return A;
  else
    return 1 + AddR2(A, B - 1);
}
```

Cette version est la plus évidente mais elle n'est pas en récursivité terminale ! En effet l'addition 1 + Addr2(…) est faite après l'appel récursif.
Si on décompose on le voit bien :

```
int AddR2bis(int A, int B)
{
  int R;
  int Rm1;

  if (B == 0)
    R = A;
  else
  {
    B = B - 1;
    Rm1 = AddR2bis(A, B);
    R = 1 + Rm1;
  }
  return R;
}
```

Intérêt de mettre en évidence une récursivité terminale

On peut l'enlever de manière automatique !

En effet, la fonction récursive :

```
F(U)
{
  if (C)
    T ;
  else
  {
    D ;
    F(a(U)) ;
  }
}
```

avec
F : fonction
C : condition d'arrêt
T : traitement à l'arrêt
D : traitement récursif
a : transformation des paramètres
(ces 2 derniers points sont souvent confondus)

peut être remplacée par la boucle :

```
while ( ! C)
{
  D ;
  U = a(U) ;
}
T ;
```

on identifie :

```
C : B == 0
T : R = A
D/a : A = A + 1 et B = B -
1
```

et on obtient :

```
int AddI1(int A, int B)
{
  int U = A;
  int V = B;
  int R;

  while (V != 0)
  {
    U = U + 1;
    V = V - 1;
  }
  R = U;
  return R;
}
```

Application à la fonction puissance du début :

```
double Puissance-Recursive(double X, int N)
{
  if (N == 0)
    return 1 ;
  else
    return X * Puissance-Recursive(X, N - 1);
}
```

Cette récursivité est elle terminale ?

Non ! En effet la multiplication de la dernière ligne s'effectue après l'appel récursif.
L'ordre d'exécution de la dernière ligne doit être compris ainsi :

```
return X * Puissance-Recursive(X, N - 1);
```

se décompose en :

```
R =  Puissance-Recursive(X, N - 1;
R = R * X;
return R;
```

La récursivité est utilisée pour mettre en place les multiplications lors de l'appel, mais ces multiplications ne seront calculées qu'après le retour de la fonction. Repousser les multiplications permet de ne pas avoir à mémoriser les résultats intermédiaires.
Pour la rendre terminale une petite transformation est nécessaire, il faut juste ajouter un argument qui va mémoriser et accumuler les produits intermédiaires. Cet accumulateur (R) devra être initialisé à 1 lors de l'appel.

```
double Puissance-Recursive2(double R, double X, int N)
{
  if (N == 0)
    return R ;
  else
    {
      R = R * X;
      N = N - 1;
      return Puissance-Recursive2(R, X, N);
    }
}
```

Cette fonction s'appelle, par exemple pour calculer 2^8, ainsi :

```
Puissance-Recursive2(1, 2, 8)
```

Notez bien l'initialisation à 1 de R.

Il est maintenant facile d'enlever la récursivité en identifiant les parties :

```
F(U)
{
  if (C)
    T ;
  else
  {
    D ;
    F(a(U)) ;
  }
}
```

```
double Puissance-Recursive2(double R, double X, int N)
{
  if (N == 0)
    return R ;
  else
    {
      R = R * X;
      N = N - 1;
      return Puissance-Recursive2(R, X, N);
    }
}
```

peut être remplacée par la boucle :

```
while ( ! C)
{
  D ;
  U = a(U) ;
}
T ;
```

on identifie :

```
C : N == 0
T : return R
D et a :
        R = R * x
    et N = N - 1
```

et on obtient :

```
double P(double X, int N)
{
  double R = 1;

  while (N != 0)
  {
    R = R * X;
    N = N - 1;
  }
  return R;
}
```

```
double Puissance_Pas_Recursive(double X, int N)
{
  double R = 1 ;
  while (N != 0)
  {
    R = R * X ;
    N = N - 1 ;
  }
}
```

ou

```
double Puissance_Iterative(double X, int N)
{
  double R = 1;

  for (;N > 0; N--)
  {
    R = R * X;
  }
  return R;
}
```

On reconnaît bien l'algorithme itératif élémentaire du calcul de X^N.

Exemple 4 : qu'est ce qu'on peut faire facilement avec de la récursivité non terminale ?

Soient les 2 fonctions suivantes, elles prennent en arguments une chaîne de caractères (de type string) et un

entier et elles épellent la chaîne de caractères en progressant avec l'entier :

```
void EpelleR1(string mot, int i)
{
  if (i == mot.length())
    return;
  else
  {
    cout << mot[i] << ':';
    EpelleR1(mot, i + 1);
  }
}
```

```
void EpelleR2(string mot, int i)
{
  if (i == mot.length())
    return;
  else
  {
    EpelleR2(mot, i + 1);
    cout << mot[i] << ':';
  }
}
```

La différence entre les 2 fonctions réside dans la position de l'appel récursif par rapport à l'affichage. La première est en **récursivité terminale** et la seconde en **récursivité non terminale**

À l'exécution on obtient, par exemple sur la chaîne « Bonjour » :

```
EpelleR1("Bonjour", 0)

B:o:n:j:o:u:r:
```

```
EpelleR2("Bonjour", 0)

r:u:o:j:n:o:B:
```

Si il est aisé de suivre le comportement de la première fonction, pour la seconde il faut bien comprendre le rôle joué par l'argument i : il est passé par valeur, c'est à dire qu'il en existe un exemplaire pour chaque appel de la fonction et c'est lui qui conserve la position du caractère à afficher lorsque l'appel récursif sera terminé.

Évidemment il est facile d'écrire une version de la première fonction sans récursivité, qu'on applique la méthode précédente ou qu'on le fasse directement on arrive à une version qui utilise une boucle qui parcours la chaîne du début à la fin en affichant les caractères rencontrés dans l'ordre.

Pour la seconde fonction le problème est différent. Enlever la récursivité dans ce cas est compliqué, il faudrait reproduire le stockage des différents indices comme cela est fait automatiquement par la récursivité…
Mais en fait, comme il est possible d'accéder à n'importe quel caractère d'une *string*, il suffit de la parcourir de la fin au début.

Souvent il est plus facile de changer d'algorithme pour enlever une récursivité, surtout si elle n'est pas terminale !

Un autre exemple, 2 fonctions très proches, seule la place de l'appel récursif change :

```
void T1(int N)
{
  if (N == 0)
    return;
  T1(N - 1);
  etoiles(N);
}

void T2(int N)
{
  if (N == 0)
    return;
  etoiles(N);
  T2(N - 1);
}
```

La fonction etoile(int N) affiche une ligne de N étoiles.

La fonction T1 va afficher un triangle pointe en haut et la 2e va afficher un triangle pointe en bas :

```
T1(10) :

*
**
***
****
*****
******
*******
********
*********
**********

T2(10) :

**********
*********
********
*******
******
*****
****
***
**
*
```

Exemple 5 : les tours de Hanoï

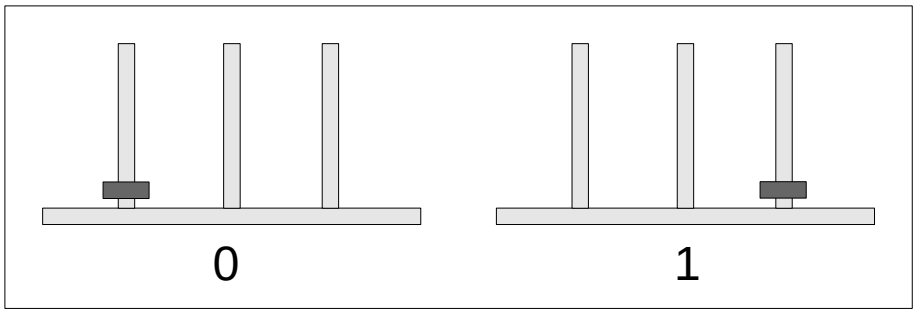
(http://fr.wikipedia.org/wiki/Tours_de_Hanoi)

Il s'agit d'un jeu de réflexion, basé sur un plateau possédant 3 piliers et N disques percé de tailles différentes. Au départ les disques sont enfilés sur le pilier de départ, en ordre croissant de haut en bas, et le but du jeu est de les déplacer sur le pilier d'arrivée en s'aidant du pilier intermédiaire tout en respectant les 2 règles :

- on ne peut déplacer qu'un disque à la fois,
- on ne peut placer un disque que sur un disque plus grand ou sur un pilier vide.

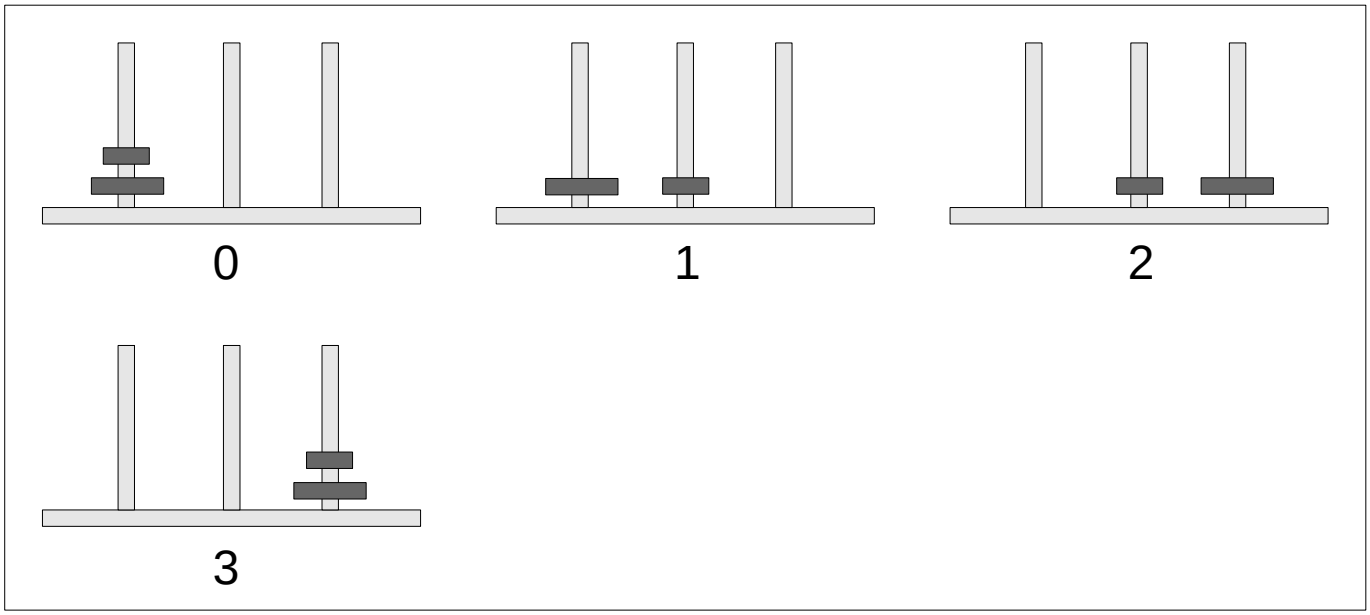
Si les solutions sont évidentes avec 1 ou 2 disques, à partir de 3 disques on voit apparaître une solution qui fait appel à la solution précédente :

- une tour constituée d'un seul disque se déplace en un seul mouvement (Dessin 1),



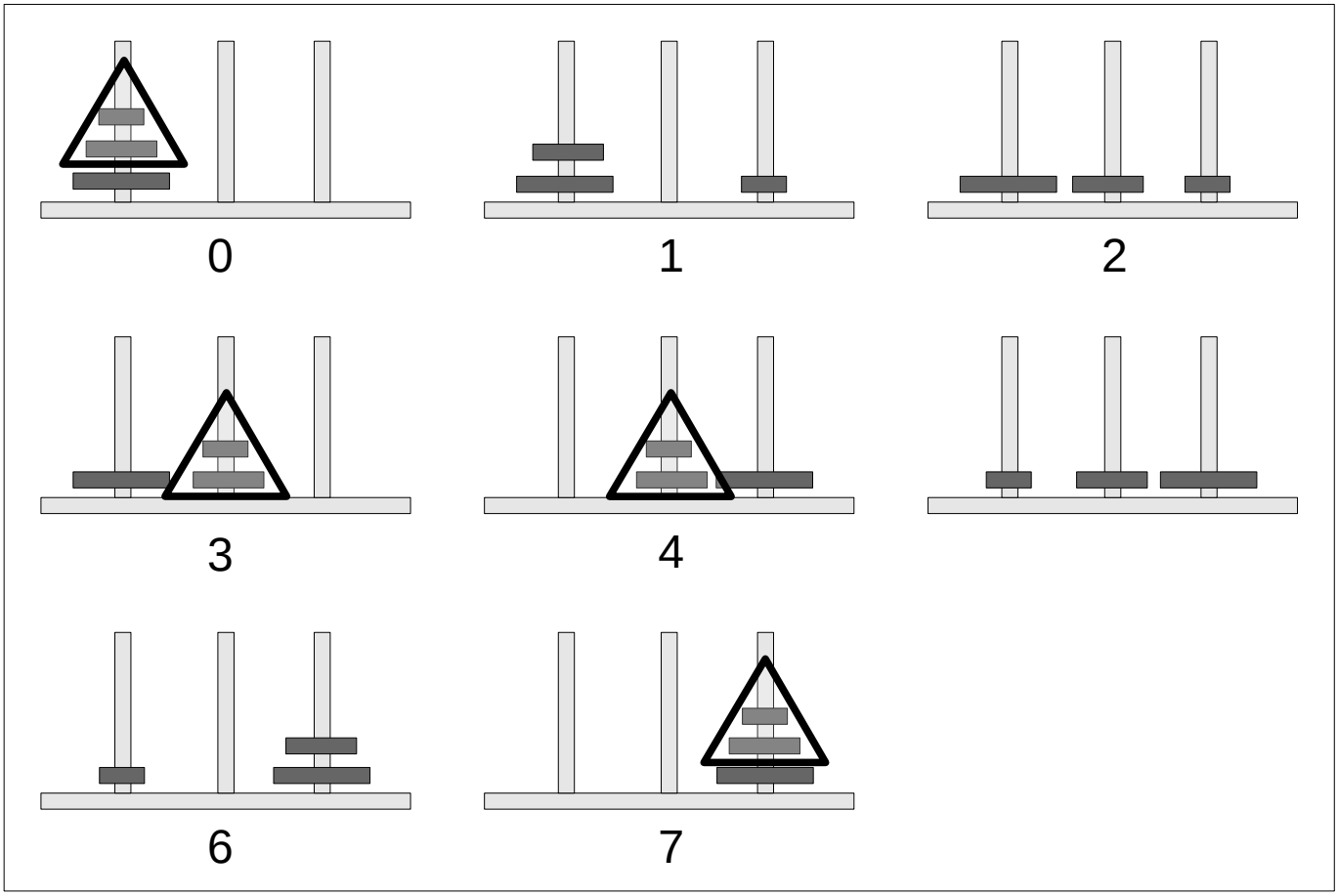
Dessin 1: Tour de Hanoï – Solution pour N = 1

- la résolution du problème pour une tour constituée de 2 disques met en évidence l'utilité du pilier intermédiaire (Dessin 2),



Dessin 2: Tour de Hanoï – Solution pour N = 2

- la résolution du problème pour N = 3 met en évidence la solution récursive. Déplacer N disques revient à
  - déplacer les N – 1 disques les plus petits sur le pilier intermédiaire (étape 1, 2, 3) en utilisant le pilier d'arrivée initial comme intermédiaire,
  - déplacer le disque le plus gros sur le pilier d'arrivée (étape 4),
  - déplacer les N – 1 disques les plus petits sur le pilier intermédiaire (étape 5, 6, 7) en utilisant le pilier de départ comme intermédiaire.



Dessin 3: Tour de Hanoï – Solution pour N = 3

La programmation d'une fonction de résolution du problème des tours de Hanoï pour N disques découle tout naturellement de cette définition récursive :

- N : nombre de disques
- depart : numéro du pilier de départ
- arrivee : numéro du pilier d'arrivée
- intermediaire : numéro du pilier intermédiaire

La fonction se contente d'afficher les déplacement en nommant les disques par leur numéro d'ordre (1 le plus petit, N le plus grand).

```
void Hanoi(int N, int depart, int arrivee, int intermediaire)
{
  if (N != 0)     // s'il y a des disques a deplacer
  {
    Hanoi(N - 1, depart, intermediaire, arrivee) ;
    cout << "Disque " << N << " : " << depart << " --> " << arrivee << endl;
    Hanoi(N - 1, intermediaire, arrivee, depart);
  }
}
```

En numérotant 1 le pilier de départ, 2 le pilier intermédiaire et 3 le pilier d'arrivée, l'appel de la fonction pour résoudre le problème à 3 disques est :

```
  Hanoi(3, 1, 3, 2) ;
```

et produit :

```
$ ./hanoi
Disque 1 : 1 --> 3
Disque 2 : 1 --> 2
Disque 1 : 3 --> 2
Disque 3 : 1 --> 3
Disque 1 : 2 --> 1
Disque 2 : 2 --> 3
Disque 1 : 1 --> 3
$
```

Quelle récursivité dans le problème des tours de Hanoï ?

On voit bien dans la fonction Hanoï qu'il y a 2 appels récursif. Le premier appel constitue bien évidemment une récursivité non terminale, mais le 2e appel correspond bien àune récursivité terminale. Il est donc facile de l'enlever pour arriver à une solution hybride avec un appel récursif pour la première moitié du problème et une boucle pour le deuxième moitié :

```
void Hanoi2(int N, int depart, int arrivee, int intermediaire)
{
  int tmp;

  while (N != 0)
    {
      Hanoi2(N - 1, depart, intermediaire, arrivee) ;
      cout << "Disque " << N << " : " << depart << " --> " << arrivee << endl;

  //  Hanoi(N - 1, intermediaire, arrivee, depart);
  //  Dans cet appel on echange juste depart avec intermediaire,
  //  il suffit donc de faire pareil
  //  tout en decrementant N :

      N = N - 1;
      tmp = depart;
      depart = intermediaire;
      intermediaire = tmp;
    }
}
```